

# Agentic AI Lab Manual

## Build Your First ReAct Agent (Pure Python, No LangChain)

Student Lab Manual | Google Colab Ready | 90 Minutes

### What You Will Learn

In this lab you will build a ReAct agent from scratch using only Python and a free LLM API. By the end, you will understand how agents think (Thought), act (Action), and learn from results (Observation).

### ALL TASKS YOU NEED TO PERFORM (Read This First)

Complete these 6 tasks in order. Each task builds on the previous one. Check off as you go.

| Task #        | What to Do                                                                                                                                             | Deliverable                                            |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Task 0</b> | Setup in Google Colab<br>- Upload notebook<br>- Install groq + duckduckgo-search<br>- Get free API key from console.groq.com and enter with getpass()  | Colab notebook running, API key set                    |
| <b>Task 1</b> | Hello LLM<br>- Call Groq Llama 3.3 70B model<br>- Ask: 'What is ReAct in one sentence?'                                                                | Screenshot of LLM response                             |
| <b>Task 2</b> | Create Tools<br>- Implement search(query) using DuckDuckGo<br>- Implement calculator(expression) for math                                              | Code for 2 tools + test search('Burj Khalifa builder') |
| <b>Task 3</b> | Write ReAct Prompt<br>- Write system prompt that forces format:<br>Thought: ... Action: tool[...]<br>Observation: ...<br>- Include Finish[answer] rule | System prompt text                                     |

|               |                                                                                                                                                                                                       |                                                                    |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Task 4</b> | Build ReAct Loop (Main Task)<br>- Write loop: call LLM, parse Action, run tool, append Observation<br>- Run on: 'Who built Burj Khalifa? What is 25*4?'<br>- Observe Thought/Action/Observation trace | Full trace screenshot showing Thought, Action, Observation, Finish |
| <b>Task 5</b> | Pakistan Challenge<br>- Run agent on: 'When was Minar-e-Pakistan built? How many years ago from 2026? Use calculator[2026-year]'                                                                      | Final answer + trace                                               |
| <b>Task 6</b> | Custom Tool (Bonus)<br>- Create your own tool e.g., pakistan_info, get_cgpa, weather<br>- Add to TOOLS dict and test                                                                                  | Code of custom tool + trace showing it was used                    |

### Submission Checklist

Before submitting, make sure you have:

1. Colab link with outputs visible (Share > Anyone with link)
2. Screenshot of Task 4 trace (Thought/Action/Observation)
3. Answer for Task 5
4. Code for Task 6 if attempted

## Part 0: Setup in Google Colab (5 minutes)

You do NOT need to install anything on your laptop. Colab runs in browser.

1. Open [colab.research.google.com](https://colab.research.google.com)
2. File > Upload notebook > Choose ReAct\_No\_LangChain\_Colab.ipynb
3. Runtime > Run all
4. First cell will run: `!pip install groq duckduckgo-search`
5. Second cell will ask for API key. Get free key from [console.groq.com](https://console.groq.com) (sign up, Create API Key). Paste it - it will be hidden.
6. You are ready.

If Groq gives rate limit error (429), change MODEL to 'llama-3.1-8b-instant' in code - it has higher free quota.

## Part 1: What is ReAct? Quick Recap

LLMs alone hallucinate and cannot use tools. ReAct solves this by looping:

**Thought:** Your reasoning - what do I need to do next? Do I have enough info?

**Action:** Call a tool in exact format: `search[query]` or `calculator[expression]` or `Finish[answer]`

**Observation:** Result from tool - real data from world, not generated by LLM

Loop: Thought -> Action -> Observation -> Thought -> ... -> Finish. This is how ChatGPT browsing and all agents work.

Example trace:

Question: Who built Burj Khalifa?

Thought: I need to find builder first

Action: `search[Burj Khalifa builder]`

Observation: Developed by Emaar Properties

Thought: I now know answer

Action: `Finish[Emaar Properties]`

## Part 2: Task 1 & 2 - Hello LLM and Tools

### Task 1: Call LLM

```
from groq import Groq
client = Groq(api_key=your_key)
resp = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "What is ReAct in one sentence?"}]
)
print(resp.choices[0].message.content)
```

### Task 2: Create Tools - These are plain Python functions

```
def search(query: str) -> str:
    from duckduckgo_search import DDGS
    with DDGS() as ddgs:
        results = list(ddgs.text(query, max_results=3))
        return "\n".join([r['body'][:200] for r in results])

def calculator(expr: str) -> str:
    return str(eval(expr))

TOOLS = {"search": search, "calculator": calculator}
```

```
# Test
print(search("Burj Khalifa builder"))
print(calculator("2026-1968"))
```

Why two tools? search for facts, calculator for reliable math. LLM is bad at math, so we give it a calculator.

## Part 3: Task 3 & 4 - Build the Agent (Main Part)

### Task 3: System Prompt - This tells LLM how to behave as ReAct agent

```
REACT_PROMPT = """
You are a ReAct agent. You have tools:
1. search[query]: Search web for facts
2. calculator[expression]: For math like calculator[182.63*1.5]
3. Finish[answer]: Give final answer

Use EXACT format:
Thought: your reasoning
Action: search[query] OR calculator[expr] OR Finish[answer]
Observation will be given after Action
... repeat ...
Thought: I now know final answer
Action: Finish[your answer]
"""
```

### Task 4: The Loop - The whole agent is ~30 lines

```
import re

def react_agent(question):
    messages = [
        {"role": "system", "content": REACT_PROMPT},
        {"role": "user", "content": question}
    ]
    for step in range(6):
        resp = client.chat.completions.create(model="llama-3.3-70b-versatile", messages=messages)
        text = resp.choices[0].message.content
        print(text) # Shows Thought and Action

        m = re.search(r"Action:\s*(\w+)\s*\[(.*?)\]", text, re.DOTALL)
        if not m: continue
        tool_name, tool_input = m.groups()

        if tool_name == "Finish":
            print(f"FINAL: {tool_input}")
            return tool_input

        observation = TOOLS[tool_name](tool_input)
        print(f"Observation: {observation}\n")

        messages.append({"role": "assistant", "content": text})
        messages.append({"role": "user", "content": f"Observation: {observation}"})

# Run it
react_agent("Who built Burj Khalifa? What is 25*4? Use calculator")
```

What to look for: Your output MUST show Thought, Action, Observation lines. If you see that, you built a real agent.

## Part 4: Task 5 & 6 - Your Challenges

### Task 5: Pakistan Challenge (Required)

Question: When was Minar-e-Pakistan built? How many years ago from 2026?

```
react_agent("When was Minar-e-Pakistan built? How many years ago from 2026? Use calculator[2026 - ye  
Expected: It should search for built year (1960-1968), then calculate 2026 - 1968 = 58 years. Screenshot the trace.
```

### Task 6: Custom Tool (Bonus, but highly recommended)

Create your own tool. Example:

```
def pakistan_info(topic: str) -> str:  
    data = {  
        "minar": "Minar-e-Pakistan built 1960-1968 in Lahore for Lahore Resolution 1940",  
        "karachi": "Karachi largest city, ~20M people"  
    }  
    return data.get(topic.lower(), "No info")
```

```
TOOLS["pakistan_info"] = pakistan_info  
# Also add to prompt: 3. pakistan_info[topic]
```

```
react_agent("Tell me about Minar using pakistan_info tool")  
Ideas for custom tools: get_student_info[roll_no], get_weather[city], translate[urdu phrase],  
get_cgpa[credits]
```

## Troubleshooting

| Problem                          | Solution                                                                                    |
|----------------------------------|---------------------------------------------------------------------------------------------|
| <b>Groq error 429 rate limit</b> | Change MODEL to 'llama-3.1-8b-instant' - higher free limit                                  |
| <b>Search returns no results</b> | Wait 20 sec and retry. DuckDuckGo sometimes blocks Colab. Try query with fewer words        |
| <b>Action parsing fails</b>      | Check your prompt format. Action must be exactly search[...] not Search or search:          |
| <b>Calculator error</b>          | Only use numbers and +-*/(. Example: calculator[2026-1968] not calculator[age is 2026-1968] |
| <b>Colab pip error</b>           | Runtime > Restart runtime after first cell                                                  |

## What to Submit

1. Colab share link: In Colab, Share > Anyone with link > Copy link (make sure outputs are visible)
2. Screenshot 1: Full trace of Task 4 (must show Thought, Action, Observation, Finish)
3. Screenshot 2: Task 5 answer
4. If you did Task 6: Code of custom tool + trace

File to upload for this lab: ReAct\_No\_LangChain\_Colab.ipynb (pure Python version only)

You have built your first Agentic AI system from scratch without any framework. This same loop powers ChatGPT browsing, Copilot, and all modern agents. Well done!